Information Processing and Retrieval Project

CATARINA GONÇALVES* and ISABEL COSTA*, ist1103989 and ist1106868

1 Functions

To address the information retrieval tasks of keyword extraction and text summarization, we developed a modular Python pipeline. Our system separates distinct operations into specialized modules dedicated to indexing, summarization, keyword extraction, and evaluation. Additionally, an Exploratory Data Analysis (`EDA.py`) was conducted, the detailed findings of which are located in the supplementary additional results page.

For the main system, `indexing_module.py` handles text processing and the construction of hybrid (lexical and semantic) indexes, while `compute_scores.py` calculates the underlying relevance metrics, for the respective index types. The `summarization_module .py` computes several summarization approaches (extractive, abstractive, MMR, or RRF). Finally, `keyword_module.py` manages keyword extraction, and `evaluation_module.py` provides the necessary evaluation metrics.

The complete workflow is executed through a Jupyter Notebook, which orchestrates the pipeline and generates the quantitative results presented in this report.

1.1 `indexing(D,args)`

In order to apply indexing, the documents must be pre-processed first. The `process_text` function acts as a custom NLP tokenizer, being responsible for cleaning the raw text and extracting meaningful vocabulary. It captures both individual words and cohesive noun phrases, while preventing term redundancy. The function processes the text through the following sequence of operations:

- **Initial Normalization:** The input text is lowercased, and irregular spacing or special characters are standardized into single spaces using regular expressions.
- **Noun Phrase Extraction:** The text is processed via the `spaCy` pipeline to capture multi-word semantic units (e.g., preserving "machine learning" instead of splitting it). Within each recognized noun chunk, tokens are filtered to exclude stop-words and non-alphabetic characters. We also enforce a length constraint (lemmas must have more than 1 character). This was added during the keyword extraction phase after observing that alphanumeric terms (e.g., "100m" in document 1 in the *Business* category) were leaving behind meaningless, single-letter artifacts ("m"), that would otherwise introduce noise into our vocabulary and distort TF-IDF weighting. Furthermore, the noun phrases were limited to a maximum of two valid words, a decision justified in our Exploratory Data Analysis (see Section [A.1.1]). Finally, the valid lemmas are concatenated with underscores into a single cohesive term.
- **Index Tracking:** To prevent double-counting words that are already part of a noun phrase, their original document indices are recorded in a `chunk_token_indices` set.
- **Individual Word Extraction:** The function iterates through the remaining individual words in the document, employing `chunk_token_indices` set to skip any token already processed. The remaining tokens are subjected to strict filtering (no stop-words, punctuation, non-alphabetic, or single-letter artifacts) before their lemmatized forms (e.g., converting "running", "runs", and "ran" down to "run") are added to the final token list. This means the TF-IDF matrix will be rather small (saving memory) and more statistically dense (improving term frequency scores).

After pre-processing, the corpus must be transformed into a searchable data structure. The `indexing` function is responsible for processing the corpus into a structured hybrid index. The indexing process is divided into the following key operations:

- **Index Granularity:** The function accepts a `level` argument to determine what size of text is being indexed. It can either index entire documents at once (which is useful for keyword extraction), or it can use `nltk.sent_tokenize` to split the documents and index them sentence-by-sentence (which is required for extractive summarization).
- **Lexical Indexing:** The required inverted index component is implemented as a sparse matrix representation through Scikit-Learn's `TfidfVectorizer`. This vectorizer is applied directly to the normalized vocabulary generated by the `process_text` tokenizer.
- **Semantic Indexing:** The required vector index is constructed using the `SentenceTransformer` library to compute dense embeddings. The function loops through the selected pre-trained language models and generates the corresponding semantic representations for the indexed texts. The two light-weight encoders selected were `all-MiniLM-L6-v2` [Wang et al. 2020] (a highly efficient, general-purpose similarity model) and `BAAI/bge- -small-en-v1.5` [Xiao et al. 2023] (a model strictly optimized for information retrieval and search tasks).
- **Required Profiling Outputs:** The function outputs three specific elements: the compiled hybrid index I, the total indexing execution time (in seconds), and the combined in-memory size of the index structures (in MB).

1.2 `summarization(d,p,l,I,args)`

The `summarization` function is the central routing engine for our system. Depending on the `summary_mode` provided in the arguments, it delegates the document to the appropriate specialized algorithm. The implementation addresses the requested behaviors and outputs through the following operations:

- **Baseline Summarization:** The `summarization_baseline` function acts as a naive evaluation benchmark. It segments the document text into sentences using `nltk.sent_tokenize` and extracts the first p sentences, assigning them a uniform default score of 1.0. This approach relies on the common structure of journalistic texts (such as the BBC News corpus),

*Both authors contributed equally to this research.

operating under the assumption that the most important information is presented at the beginning of the article.

- **Lexical Summarization:** This extractive method ranks sentences by measuring their lexical similarity to the source document. Utilizing the `compute_lexical_scores` function, it implements the Okapi BM25 ranking function ($k_1 = 1.2$ and $b = 0.75$). To maximize computational efficiency, the entire document text is processed into a `set` of unique query terms. Each candidate sentence is evaluated by applying `process_text` to compute the Term Frequency (TF), retrieving the corresponding Inverse Document Frequency (IDF) directly from the Scikit-Learn matrix, and applying a length penalty based on the corpus average (`avgdl`). Lastly, the top p scoring sentences are extracted.
- **Semantic Summarization:** This extractive method ranks sentences based on their semantic similarity to the source document, rather than exact keyword overlap. It uses the `compute_semantic_scores` function, which retrieves the dense embeddings produced by the encoder LLMs during the indexing phase. To score the text, the algorithm first constructs a "Document Vector" by taking the mathematical average (`np.mean`) of all its sentence vectors. It then applies Scikit-Learn's `cosine_similarity` to calculate the multi-dimensional angle between each sentence and the document vector, extracting the top p sentences with the highest semantic alignment.
- **Handling Redundancy:** A notable problem with selecting sentences of high informational value is the potential redundancy among them. To increase sentence diversity, the `mmr_summarization` function implements Maximal Marginal Relevance (MMR). First, the algorithm normalizes the baseline relevance scores (either lexical or semantic) to a $[0.0, 1.0]$ scale. It then selects the next most informative sentence to add to the summary set S, removing it from the candidate pool. The selection attempts to simultaneously maximize relevance to the document d and dissimilarity to the previously selected sentences v, dictated by the following equation:

$$MMR(s) = (1 - \lambda) \times sim(s, d) - \lambda \sum_{v \in S} sim(s, v)$$

  where sim denotes the cosine similarity. To optimize the performance of this iterative loop, our implementation reuses the pre-computed semantic vectors from the hybrid index, which avoids redundant text re-encoding while calculating the required similarities.
- **Ensembling:** To evaluate the consensus between sparse and dense representations, the `rrf_summarization` function implements Reciprocal Rank Fusion (RRF), a hybrid ensemble approach. The algorithm calculates a fused score for each candidate sentence s across the compared approaches F using the designated equation:

$$RRF(s) = \sum_{f \in F} \frac{1}{\mu + rank(f(s))}, \quad \text{with } \mu = 5$$

  where, $rank(f(s))$ represents the ordinal rank of the sentence under a specific approach f. Sentences that perform consistently well across both retrieval paradigms receive the highest fused RRF scores, and are extracted to form the final summary.
- **Generative Summarization:** To explore generative design options, the system shifts focus to prompting pretrained decoder LLMs to perform abstractive summarization. This method utilizes two lightweight autoregressive LLMs sourced from HuggingFace, with intentionally distinct profiles: `TinyLlama` [Zhang et al. 2024] (instruction-tuned conversational architecture) and `distilgpt2` [Sanh et al. 2019] (base continuation model). To guarantee conformative outputs, two distinct prompting strategies were included, tailored to each model's specific learning setup. For `TinyLlama`, it constructs structured system messages assigning the persona of a "professional journalist" and formats them using the tokenizer's native `apply_chat_template`. For the `distilgpt2` model, it utilizes a classical continuation prompt by appending `"TL;DR:"` to the source text. To prevent context-window overflow and memory errors, the source document is safely truncated before generation, and the final output is returned as a raw text string.

1.3 `keyword_extraction(d,p,I,args)`

1.3.1 Extractive Keyword Extraction.

We implemented an extractive keyword extraction system based on the TF-IDF vectorizer previously utilized in our indexing phase. The detailed properties and algorithmic justifications for this approach (namely the length-based token filtering and the index-based mechanism designed to prevent constituent words of a noun phrase from being redundantly scored as standalone unigrams) are thoroughly discussed in Section [1.1].

1.3.2 Abstractive Keyword Extraction.

For the abstractive keyword generation phase, our pipeline retrieves the source document and truncates it to 1200 characters, to respect the context limits of lightweight models. Since LLMs possess distinct instruction, the text is routed through prompt templates and inference parameters specific to each model. Finally, a universal post-processing routine (lowercasing, punctuation removal, and string splitting) forces the output into a strict array of exactly p keywords, to guarantee a balanced comparison against the extractive baseline.

The specific models and their corresponding queries are detailed below:

- **`TinyLlama/TinyLlama-1.1B-Chat-v1.0`:** We selected this model to serve as a robust, lightweight baseline. It is highly regarded for its fast inference speeds and low memory footprint, which makes it ideal for processing large document batches without requiring great computational resources. For this model, we utilized a standard zero-shot instruction prompt (e.g., *"Extract exactly p core keywords from the text below. Return ONLY a comma-separated list of words."*). TinyLlama proved to be highly capable, responding effectively to direct, straightforward instructions. It did not require extensive prompt engineering or complex conversational tags to comprehend the objective. Instead, a simple text-completion query

was sufficient to reliably generate the requested comma-separated format.

- **`Qwen/Qwen2.5-0.5B-Instruct`:** This model was chosen to evaluate the keyword extraction capabilities of an ultra-lightw eight, state-of-the-art architecture. To maximize its capabilities, we implemented a highly structured prompt through strict ChatML tags (e.g., `<|im_start|>system`), to explicitly assign an "expert data extractor" persona. Furthermore, we disabled probabilistic sampling by setting `do_sample=False` during inference. Because generative models at this scale can occasionally struggle with hallucination or append unnecessary conversational filler, leveraging its native ChatML formatting alongside deterministic generation was essential.

1.4 `evaluation(`S_{set}`, `R_{set}`, args)`

The evaluation pipeline is structured to assess performance at both the individual document level, and the overall collection level.

The document level evaluation operates article-by-article by iterating through the dataset of generated summaries. For each individual document, the specific performance metrics detailed below are calculated in isolation. These granular results are then logged into a centralized dictionary.

Once the document level scoring concludes, the system shifts to a collection level evaluation by computing the macro-averages of the logged metrics. This ensures that every document contributes equally to the final system score, regardless of its length. For scalar metrics like ROUGE, Precision, or Semantic Similarity, the system calculates the standard arithmetic mean across all successfully evaluated documents. For complex array aggregations, such as the extractive 11-point interpolated Precision-Recall curve, it merges large amounts of individual document curves by computing the vertical average across the standard recall points.

1.4.1 Extractive Evaluation.

The original `BBC News Summary (Summaries)` dataset provides summaries, which cannot be directly used to evaluate the extractive model via exact index matching. To bridge this gap, an automated alignment mechanism (`build_extractive_r_set`) was implemented. This function segments the original document into individual sentences, and utilizes a SentenceTransformer model (`all-MiniLM-L6-v2`) to encode both the individual sentences and the provided summary into dense vector embeddings. By computing the cosine similarity between these embeddings, the system identifies the top p original sentences that most closely capture the semantic meaning of the provided summary, thereby synthesizing a reliable extractive ground truth (R_{set}).

Subsequently, the summaries generated by the `extractive_summ arization` function were evaluated against this synthesized ground truth through Precision, Recall, F_β measure, Average Precision, and an 11-point interpolated Precision-Recall (PR) curve, aligned with the following decisions:

- **Domain-Adaptive F_β-measure:** To align our evaluation with the distinct reading behaviors associated with different news categories, we applied a dynamic β parameter. For *Business*, *Politics*, and *Tech*, we set $\beta = 0.5$ to heavily prioritize Precision. Readers of these domains require dense, factual reporting, where irrelevant sentences (false positives) rapidly degrade the summary's utility. Conversely, for narrative-driven domains like *Entertainment* and *Sport*, we set $\beta = 2.0$ to prioritize Recall. In these contexts, capturing sequential events and commentary is very important. Omitting key highlights (false negatives) is far more detrimental to the reader's experience than including slight excess context.
- **Uninterpolated Average Precision (AP):** Given that our extractive summarization approach yields a ranked list of sentences, we evaluated the output using uninterpolated AP. Unlike standard set-based metrics, AP evaluates ranking quality by rewarding models that position the most relevant sentences at the very top of the generated summary. It is calculated as:
$$AP = \frac{\sum_{k=1}^{n} (P(k) \times rel(k))}{|R|} \tag{1}$$
where n is the total number of extracted sentences, $P(k)$ is the precision at rank k, $rel(k)$ is an indicator function equaling 1 if the sentence at rank k is relevant and 0 otherwise, and $|R|$ represents the total number of relevant sentences in the synthesized ground truth (R_{set}).
- **11-Point Interpolated PR-Curve:** To visualize the trade-off between Precision and Recall across the entire dataset, we mapped our results to an 11-point interpolated Precision-Recall curve. Interpolating precision at 11 standardized recall intervals $(0.0, 0.1, \ldots, 1.0)$ allowed us to smooth out local rank variations and establish a universal x-axis. This standardization then enabled us to compute a clean macro-average across all categories, providing a complete and stable visualization of how system precision degrades as deeper recall is enforced.

1.4.2 Abstractive Evaluation.

- **`all-mpnet-base-v2`:** To evaluate the semantic overlap between the system-generated summary (S_{text}) and the reference summary (R_{text}) both texts are passed through the `all-mpnet-base-v2` encoder. This specific SentenceTransformer was selected because it is widely recognized as one of the highest performing models for general purpose semantic text similarity.
- **Abstractive Evaluation Metrics (Cosine Similarity and ROUGE):** We utilized two metrics to assess both semantic meaning and lexical structure. Since abstractive models inherently paraphrase, exact-match metrics often unfairly penalize valid synonyms. Cosine Similarity was chosen to evaluate semantic overlap because it ensures the generated text captures the factual core of the reference, regardless of the vocabulary used. To complement this, we used ROUGE to measure structural fluency. We computed ROUGE-1 (unigrams) for core vocabulary retention, ROUGE-2 (bigrams) for phrase coherence, and ROUGE-L (longest common subsequence) to verify that the generated summaries structurally align with the references.

2 Questions

2.1 Are keywords extracted under TF-IDF criteria and decoder LLMs in agreement? Why?

- **Theory:** To answer this question, it is essential to first understand the theoretical differences in how these paradigms process text, which naturally leads to divergent outputs. TF-IDF is a purely lexical approach that extracts keywords explicitly written within the source text. Conversely, generative decoder LLMs abstractly comprehend the text and can generate overarching concepts, often categorizing documents using multi-word phrases, synonyms, or vocabulary that does not explicitly appear in the original text.
- **Jaccard and Cosine Similarity:** To rigorously evaluate the agreement between these methods, we implemented a dual-metric evaluation pipeline that captures both exact string matches and underlying meaning. For lexical agreement, we utilized Jaccard Similarity, which treats keywords as literal strings of characters and calculates the intersection over the union ($Jaccard(A, B) = \frac{|A \cap B|}{|A \cup B|}$) to determine if the exact same character strings appear in both lists. For semantic agreement, we utilized the `all-MiniLM-L6-v2` Sentence-Transformer to encode the combined keyword lists into dense, high-dimensional numerical vectors. By calculating the cosine similarity between these two vectors, the code evaluates whether the extracted themes share the same underlying meaning, regardless of the specific vocabulary used. After calculating these values for each document, we computed the arithmetic mean across the entire collection to derive the final macro-averaged results for the corpus.
- **Results and conclusions:** The macro-averaged results across a representative subset of the corpus confirm a significant divergence between the two approaches. Due to the high computational cost and prolonged inference latency associated with generative LLMs, this specific analysis was constrained to the first 100 documents of the dataset, which provided a sufficiently robust sample to establish clear behavioral trends. For the `TinyLlama` model, we observed a mean lexical agreement (Jaccard) of 9.89% compared to a significantly higher semantic agreement (Cosine Similarity) of 58.72%. Similarly, the `Qwen` model yielded a mean lexical agreement of 8.14% and a semantic agreement of 60.88%. These results indicate that while the models are in strong semantic agreement regarding the core themes of the text, they diverge significantly in their lexical choices, confirming that the LLMs prioritize conceptual synthesis over exact word-for-word extraction.

2.2 How does the extractive summarization system perform for the full collection? And within each category? Any intuition for the observed differences?

To understand the impact of extractive summarization, we evaluated the performance of the pure Lexical (BM25) system. Across the full collection, the model achieved a macro-averaged F-Beta of 0.3663 and an Average Precision of 0.2729. The behavior of this extraction is further contextualized by the 11-point interpolated precision-recall curve (Figure 1). The curve demonstrates a relatively stable precision ceiling of approximately 0.60 up to a recall of 0.2, after which it experiences a steep, stair-step decline, dropping to nearly zero by a recall of 0.7. This indicates that while the lexical model is very confident in its top ranked sentence, it struggles to exhaustively retrieve all relevant reference sentences without accumulating false positives.

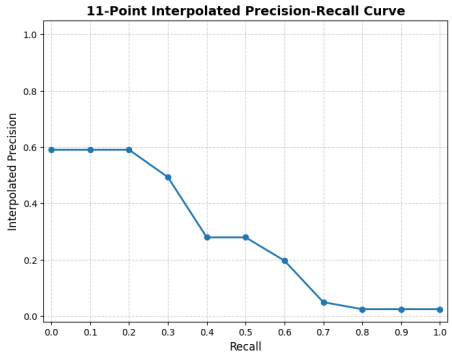


Fig. 1. 11-Points PR Curve for the lexical (BM25) summarization

Breaking down the F-Beta scores by category reveals a significant performance divergence:

- *Sport*: 0.4299;
- *Entertainment*: 0.3679;
- *Politics* - 0.3507;
- *Business* - 0.3477;
- *Tech* - 0.3236.

The intuition for these differences correlates directly with the structural characteristics identified during our Exploratory Data Analysis. The highest performing category, Sport, has a shorter median length and highly concentrated vocabulary, that creates massive term frequency overlaps, which allows the BM25 algorithm to easily isolate the most important sentences. The lowest performer category, *Tech*, has the longest and densest category in the corpus, and suffers from severe vocabulary drift and keyword dilution.
These baseline results prove that pure lexical extractive summarization is highly sensitive to document length and vocabulary concentration, requiring different approaches (such as semantic embeddings) to accurately handle longer, denser journalistic texts.

2.3 How IR models affect summaries? Are dense representations more effective than sparse ones? Which LLM encoder achieves the highest performance? Why?

The choice of the IR model fundamentally alters the quality of a summary by shifting the extraction criteria from positional heuristics to actual content relevance. When using a standard Baseline approach (no IR), which simply extracts the first few sentences by relying on the natural "lead bias" of news articles, the system achieves a respectable F_β of 0.3708 and an AP of 0.3125. Interestingly, introducing a purely lexical IR model (BM25) actually results in a slight

performance dip (F_β = 0.3663, AP = 0.2729). As visualized in the 11-Point Interpolated Precision-Recall curves, the BM25 model exhibits a sharp decline in precision as recall increases. This illustrates that forcing a summary to rely on exact keyword matching is brittle, as it can disrupt the natural flow of the lead paragraphs and miss crucial information if synonyms are used. However, transitioning to a context aware semantic IR model (`MiniLM`) completely transforms the output, driving the F_β up to a peak of 0.5321 and the AP to 0.4570. The corresponding PR curve reflects this superiority, showing that the dense semantic model successfully maintains much higher precision levels throughout the mid to high recall stages (the 0.4 to 0.6 range). This 43% improvement over the baseline shows that an IR model dramatically enhances a summary, but only if it possesses the semantic capacity to bypass brittle vocabulary mismatches and reliably extract the true conceptual core of the document.

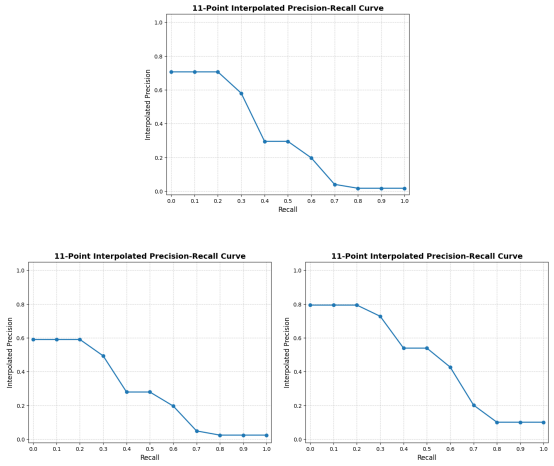


Fig. 2. 11-points PR curves for Baseline, BM25, and MiniLM respectively

The empirical data confirms that dense representations are significantly more effective than sparse ones for document summarization. Transitioning from the sparse BM25 model to the dense encoders resulted in substantial performance leaps. The F_β score went from 0.3663 (BM25) to 0.4834 with the BAAI encoder, and reached a peak of 0.5321 with the MiniLM encoder, a 45% improvement over the lexical baseline. This is most visible when examining the 11-Point Interpolated Precision-Recall curves presented in Figure 3. The BM25 curve exhibits a sharp decline toward zero as recall increases, indicating that its precision fails rapidly when attempting to retrieve more than a few sentences. On the other hand, both dense models maintain much higher precision levels at the mid to high recall stages (the 0.4 to 0.6 range). This demonstrates that dense vectors are far more robust at maintaining topical relevance throughout the entire length of a document compared to traditional keyword based methods.

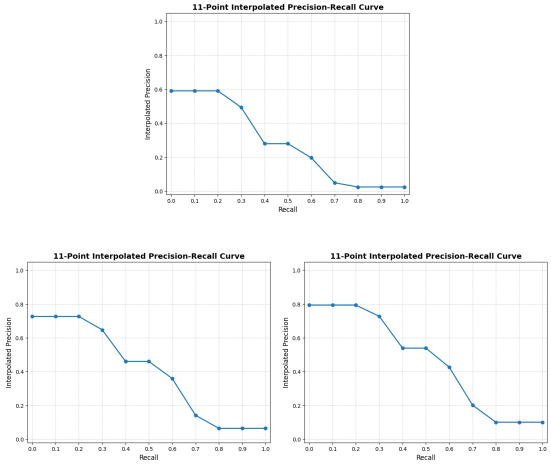


Fig. 3. 11-points PR curves for BM25, BAAI and MiniLM respectively

Among the evaluated models, the `all-MiniLM-L6-v2` encoder unequivocally achieves the highest performance. It reaches a peak F-Beta of 0.5321 and an AP of 0.4570, significantly outperforming the alternative dense BAAI encoder, which stalled at an F-Beta of 0.4834 and an AP of 0.3940. The primary reason for this superiority lies in its specific architectural optimization. While many LLMs are trained for general purpose text generation or comprehension, `MiniLM` is specifically fine-tuned for Semantic Textual Similarity tasks. Its training objective is focused on mapping sentences with identical underlying meanings to the exact same region of a vector space. Consequently, when the extraction pipeline attempts to align sentences from the source document to the synthesized abstractive summary, `MiniLM` is significantly more accurate at recognizing semantic equivalence despite drastic vocabulary changes. This makes it highly robust across varied vocabularies, as evidenced by its dominant F-Beta scores in complex and jargon-heavy categories like *Business* (0.5745) and *Politics* (0.5564).

2.4 Considering MMR, how λ impacts the accuracy (against reference extracts) of summaries? Should λ be a fixed threshold or depend on the provided topic document (d-specific)?

Evaluating the semantic MMR model across varying diversity penalties ($\lambda \in \{0.2, 0.5, 0.8\}$) yielded completely identical accuracy metrics globally (Macro F-Beta: 0.2422, AP: 0.2116) and across all individual categories. This static performance reveals a disconnect between internal redundancy and reference accuracy. While higher λ values successfully diversify the generated summary, these compositional shifts do not improve F-Beta scores because the reference summaries possibly prioritize relevance over diversity.

To understand these compositional shifts, we analyzed summary redundancy via cosine similarity. Although a baseline threshold of $\lambda = 0.5$ successfully reduced overall corpus redundancy by 18.2%, the document level analysis reveals high volatility.

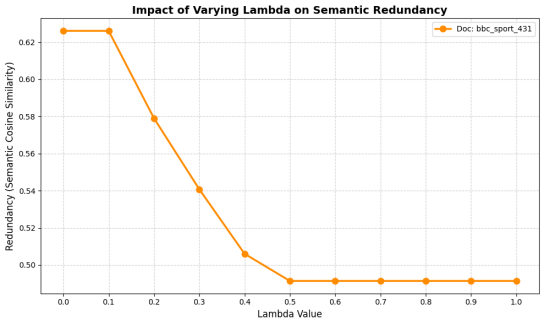


Fig. 4. Impact of varying λ on redundancy for a single document.

A random document was selected and the variation of λ was analyzed. As shown in Figure 4, for the document bbc_sport_431, redundancy decreases sharply between $\lambda = 0.1$ and $\lambda = 0.5$, after which it plateaus. This demonstrates that the diversity penalty fully dominates the selection process, the algorithm extracts the most divergent sentences available in the candidate pool, and further increasing the λ penalty yields the same summary and no additional reduction in redundancy.

Given the prohibitive nature of conducting an analysis across the entire corpus of 2,225 documents, we aggregated the redundancy metrics by category to identify broader structural trends.

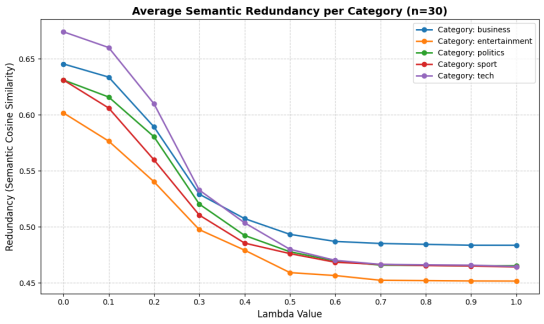


Fig. 5. Average redundancy (cosine similarity) per news category as a function of the MMR penalty λ (n=30 documents per category).

The trends regarding categories observed in Figure 5 align with our structural analysis of the corpus. *Tech* documents, which our EDA confirmed as one of the longest category on average, exhibit the highest baseline redundancy at $\lambda = 0.0$. In contrast, categories with shorter average lengths, such as *Entertainment* and *Sport* exhibit naturally lower initial redundancies.

As the penalty increases, all categories experience reduction in redundancy up to $\lambda = 0.5$. However, this reduction does not continue indefinitely. Instead, every category reaches a strict plateau by $\lambda = 0.7$. The visual evidence also highlights that different categories plateau at slightly different redundancy floors (e.g., *Business* maintains a noticeably higher floor than the other four categories). This behavioral divergence shows that MMR penalties are highly sensitive to text length and internal density. Therefore, rather than utilizing a fixed threshold, a document-specific λ, potentially weighted by the document's sentence count seems most appropriate.

2.5 Under embedding-based quality stances, is abstractive summarization superior to extractive summarization? Why?

In theory, abstractive summarization is often considered superior under embedding-based stances because it can synthesize concepts and avoid the rigid structural baggage of extracted sentences. However, our empirical data contradicts this, as a highly optimized dense extractive pipeline could actually outperform a small-scale abstractive generator. When evaluating our abstractive representative (`TinyLlama`), the model achieved a Semantic Similarity of 0.8042. This is a strong score, proving that despite its lower exact word overlap (ROUGE-1 of 0.4195), it successfully captured the conceptual themes of the text. Yet, our strongest extractive representative (`MiniLM`) surpassed it, achieving an outstanding Semantic Similarity of 0.8362 (alongside a dominant ROUGE-1 of 0.5526). The reason for this extractive lead lies in factual preservation and model scale. While `TinyLlama` attempts to generate new text, omitted entities, or lost nuances, the dense `MiniLM` encoder is optimized to align exact and factually dense source sentences with the reference summary space. Therefore, under an embedding-based stance, a precise semantic retriever currently yields higher quality summaries than a compact generative LLM, as it guarantees the preservation of the document's core semantic integrity without the stochastic variability of text generation.

2.6 Is Reciprocal Rank Fusion (RRF) useful to aid decisions?

Empirical results prove that RRF is highly useful for aiding extraction decisions by acting as a voting system between distinct retrieval approaches. Sparse models (BM25) are great at exact keyword matches but lack deep context, while dense models (LLM embeddings) understand context but occasionally miss crucial proper nouns. Furthermore, because BM25 generates unbounded numerical scores and semantic models generate strictly bounded cosine similarities, their raw outputs are mathematically incompatible. RRF bypasses this by translating raw scores into ordinal placements.

While the pure Semantic model achieved the highest global Macro F-Beta (0.4834) compared to the Lexical (0.3663), RRF achieved a F-Beta of 0.4752 while achieving the highest overall Average Precision (0.3965). This indicates that RRF successfully borrows the strongest signals from each paradigm. As illustrated by the RRF Precision-Recall curve 6, this consensus approach successfully maintains higher precision at deeper recall levels.

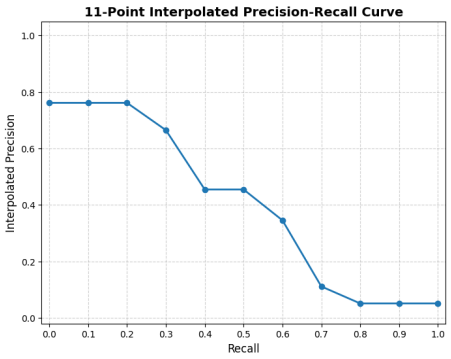


Fig. 6. 11-points PR curves for the RRF

Our manual inspection of individual documents, such as `bbc_politics_105`, reveals exactly how this voting system aids behavior in practice:

- Sentence [23] was highly prioritized by both baselines (Semantic Rank 1, Lexical Rank 2), leading RRF to elevate it to the absolute top of the combined summary.
- Sentence [18], containing critical statistical figures, was ranked 3rd (Lexical) and 4th (Semantic). RRF successfully fused these moderate agreements to boost it to the 2nd overall position.
- RRF safely retained the strongest distinct signals from isolated representations, extracting sentence [10] (Lexical's top choice, ignored by Semantic) and sentence [21] (Semantic's second choice, ignored by Lexical).

Ultimately, RRF is a highly effective utility. If a sentence only does well in one system, its score is penalized, ensuring the final summary is a balanced extraction.

A Supplementary Results

A.1 Exploratory Data Analysis

As a preliminary step, we conducted an Exploratory Data Analysis (EDA) to understand the structural and linguistic characteristics of the *BBC Summary News (News Articles)* dataset. Our EDA focused on two primary metrics: structural complexity of noun phrases and document length by category.

A.1.1 Dimension of noun-phrases.

The semantic core of a news article is heavily concentrated within its noun phrases. Thus, analyzing their density and distribution is critical.

To profile the linguistic complexity of the *BBC Summary News (News Articles)* dataset, we conducted a noun phrase extraction across the entire corpus. Utilizing the `spaCy` library, the algorithm parsed the complete text of every document to isolate continuous noun chunks. Rather than simply counting the total number of phrases, the function measured the exact token length of every individual noun phrase extracted. These token counts were then aggregated to calculate the distribution of phrase lengths, as illustrated in Figure 7.

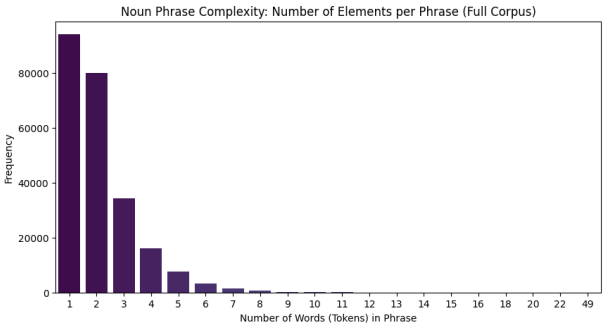


Fig. 7. Histogram of the evolution of the dimension of noun-phrases

The length of noun phrases across the corpus exhibits a severe right-skewed distribution. Unigrams and Bigrams heavily dominate the linguistic structure of the dataset, recording frequencies of approximately 95.000 and 80.000, respectively. Beyond two tokens, there is a steep and immediate decline in frequency.

Given these results, we restricted the `process_text` function to only extract and consider noun phrases with a maximum length of two tokens.

A.1.2 Optimal Summary Length per Category.

In extractive summarization, determining the optimal number of sentences (p) to extract from the source text is critical. Hence, rather than relying on an arbitrary parameter, we analyzed the natural length of the original news articles across different categories to establish an adequate criterion.

The `nltk` library allowed us to tokenize the raw text of the corpus into individual sentences, record the total sentence count per document, and group these metrics by their respective news domains, to analyze their statistical distributions. To visualize these structural variations, we plotted the histograms by category, presented in Figure 8, overlaid with their exact median values. Relying on the median rather than the mean mitigates the skew from unusually long outlier documents, and enables us to establish a dynamic extraction rule: the parameter p is calculated as 20% of a category's median length, subject to a hard baseline of three sentences, to ensure sufficient context is captured even for the briefest articles.

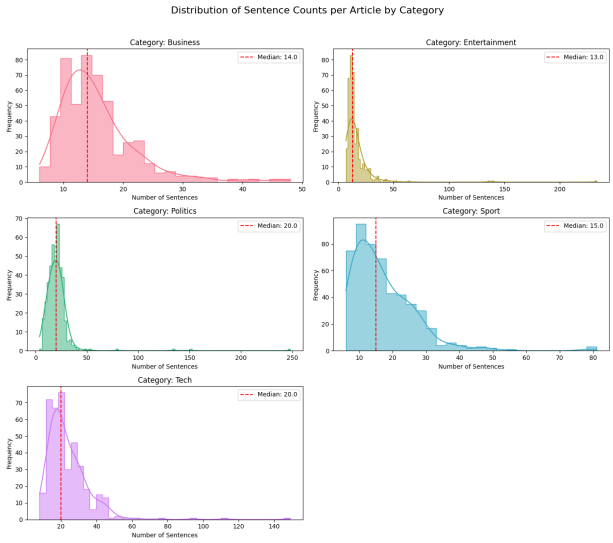


Fig. 8. Histogram of the evolution of the dimension of noun-phrases

From these plots, one notes pronounced structural differences across the categories. The computed medians established that *Politics* and *Tech* articles are naturally denser (both at a median of 20.0 sentences) compared to *Business* (14.0), *Sport* (15.0), and *Entertainment* (13.0). Consequently, we set $p = 4$ for the *Politics* and *Tech* categories, and assigned the baseline $p = 3$ for *Business*, *Entertainment*, and *Sport*.

References

Rui Henriques. 2026. Information Retrieval Course Materials. https://fenix.tecnico.ulisboa.pt/disciplinas/RGI610/2025-2026/2-semestre

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. In *NeurIPS EMC^2 Workshop*.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, Vol. 33. 5776–5788.

Shitao Xiao, Zheng Liu, Pei Zhao, et al. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. *arXiv preprint arXiv:2309.07597* (2023).

Pei Zhang, Guang Zeng, Tianrui Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. *arXiv preprint arXiv:2401.02385* (2024).